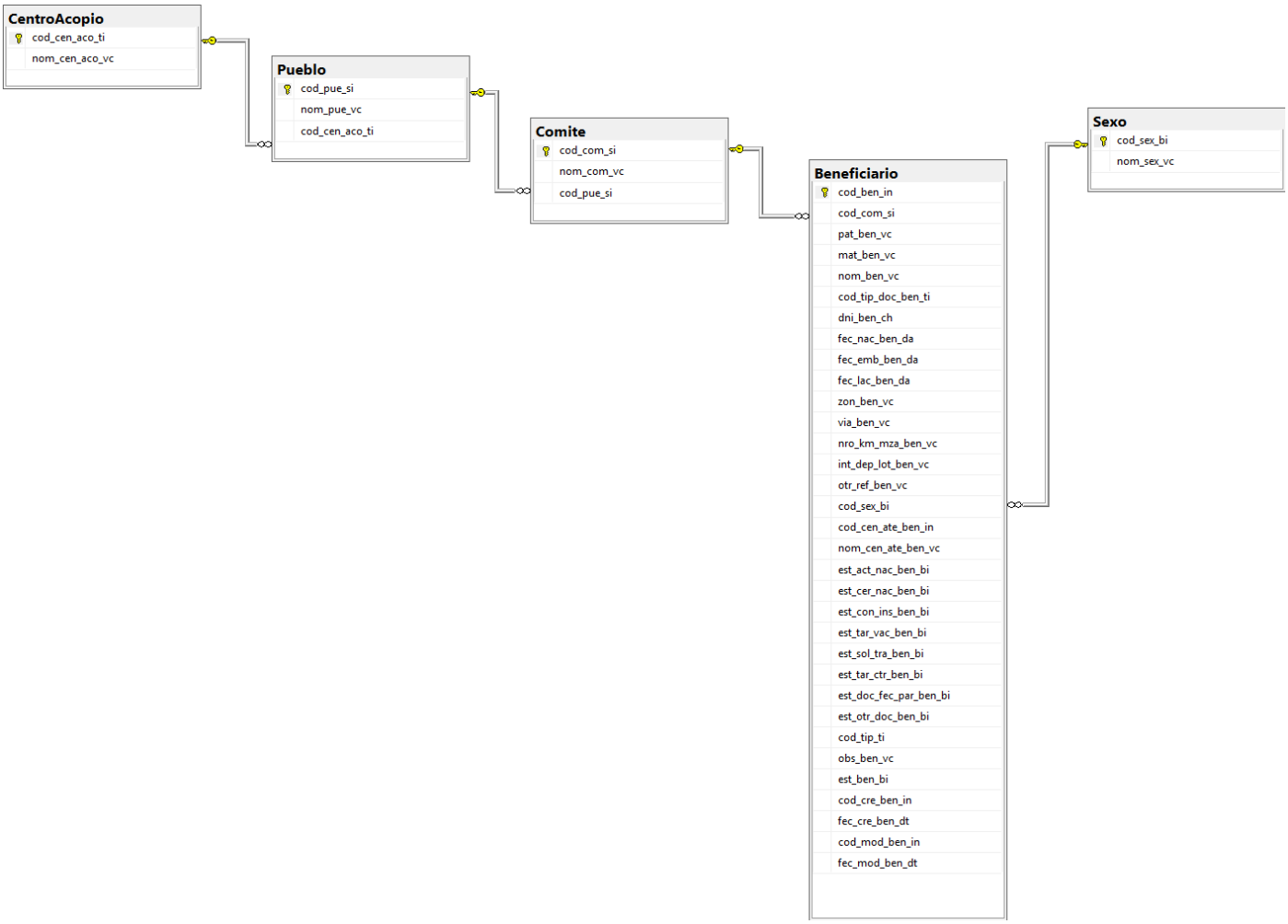
# Consultas SQL

- **DDL –Data Definition Language**
  - Lenguaje de definición de datos
    - CREATE: Crear objetos
    - ALTER: Modificar objetos
    - DROP: Eliminar objetos
- **DML –Data Manipulation Language**
  - Lenguaje de manipulación de datos
    - INSERT: Ingresar registros
    - UPDATE: Modificar los registros
    - DELETE: Eliminar registros
- **DQL –Data Query Language**
  - Lenguaje de consulta de datos
    - SELECT
- **DCL –Data Control Language**
  - Lenguaje de control de datos
    - GRANT: Otorgamos permisos
    - DENY: Denegar permisos
    - REVOKE: Revocar el permiso
- **TCC –Transactional Control Commands**
  - Comandos de control transaccional
    - BEGIN TRANSACTION: inicio
    - COMMIT TRANSACTION: Guardo los datos
    - ROOLBACK TRANSACTION: Si hay falla. Deshago todo lo modificado

## DIAGRAMA DE LA BASE DE DATOS PVL




# SELECT

La sentencia SELECT en SQL es una de las más fundamentales y potentes, utilizada para recuperar datos de una base de datos.

## Sintaxis

```
SELECT Columna1, Columna2, Columna3, .....
FROM Esquema.Tabla
WHERE Condición;
```

## Selección de todas las columnas (*)

```
USE pvl
GO
SELECT * FROM Beneficiario
GO
```

## Selección de columnas específicas

```
USE pvl
GO
SELECT cod_ben_in,
       cod_com_si,
       pat_ben_vc,
       mat_ben_vc,
       nom_ben_vc,
       dni_ben_ch,
       fec_nac_ben_da,
       fec_emb_ben_da,
       fec_lac_ben_da,
       cod_sex_bi
FROM Beneficiario
GO
```

## Alias de columnas (AS)

> - Renombrado de encabezados para reportes

```
USE pvl
GO
SELECT cod_ben_in AS CÓDIGO,
       pat_ben_vc AS PATERNO,
       mat_ben_vc AS MATERNO,
       nom_ben_vc AS NOMBRES,
       dni_ben_ch AS DNI,
       fec_nac_ben_da AS FECHA_NAC,
       fec_emb_ben_da AS FECHA_EMB,
       fec_lac_ben_da AS FECHA_LAC,
       cod_sex_bi AS SEXO
FROM Beneficiario
GO
```

| | CÓDIGO | PATERNO | MATERNO | NOMBRES | DNI | FECHA_NAC | FECHA_EMB | FECHA_LAC | SEXO |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 3 | ROSALES | RAVICHAHUA | SALOME | | 1930-10-12 | NULL | NULL | 1 |
| 2 | 4 | TUNQUI | DELGADO | VALENTIN | 08075067 | 1935-03-11 | NULL | NULL | 1 |
| 3 | 5 | CONTRERAS | FERNANDEZ | DINA LUCIA | 07437315 | 1948-05-30 | NULL | NULL | 0 |
| 4 | 7 | AYALA | CALDERON | YALITZA ANGELICA | | 2004-03-22 | NULL | NULL | 0 |
| 5 | 8 | TENA | AGUILAR | JULISSA | | 1998-05-12 | NULL | NULL | 0 |
| 6 | 9 | QUIROZ | FIESTAS | LUCIANO | 07066071 | 1932-03-02 | NULL | NULL | 1 |
| 7 | 10 | QUIROZ | SIERRA | EDU | | 2003-04-07 | NULL | NULL | 1 |
| 8 | 11 | GONZALES | BRAVO | MARCO | | 2002-06-10 | NULL | NULL | 1 |
| 9 | 13 | VIVAS | DELA CRUZ | ALISSON ELENA | | 1997-05-02 | NULL | NULL | 0 |
| 10 | 14 | BARREDA | RAMIREZ | JESSENIA | | 1993-12-18 | NULL | NULL | 0 |
| 11 | 15 | RIVAS | VALLE | JUSTA | | 1943-01-08 | NULL | NULL | 0 |
| 12 | 17 | LAZARO | REYNOSO | BERNABE | 10580144 | 1901-01-01 | NULL | NULL | 1 |

```
USE pvl
GO
SELECT cod_ben_in AS 'CÓDIGO DEL BENEFICIARIO',
       pat_ben_vc AS 'APELLIDO PATERNO',
       mat_ben_vc AS 'APELLIDO MATERNO',
       nom_ben_vc AS 'NOMBRES DEL BENEFICIARIO',
       dni_ben_ch AS 'DNI DEL BENEFICIARIO',
       fec_nac_ben_da AS 'FECHA DE NACIMIENTO',
       fec_emb_ben_da AS 'FECHA DE INICIO DE EMBARAZO',
       fec_lac_ben_da AS 'FECHA DE INICIO DE LACTANCIA',
       cod_sex_bi AS 'SEXO DEL BENEFICIARIO'
FROM Beneficiario
GO
```

```
USE pvl
GO
SELECT cod_ben_in AS [CÓDIGO DEL BENEFICIARIO],
       pat_ben_vc AS [APELLIDO PATERNO],
       mat_ben_vc AS [APELLIDO MATERNO],
       nom_ben_vc AS [NOMBRES DEL BENEFICIARIO],
       dni_ben_ch AS [DNI DEL BENEFICIARIO],
       fec_nac_ben_da AS [FECHA DE NACIMIENTO],
       fec_emb_ben_da AS [FECHA DE INICIO DE EMBARAZO],
       fec_lac_ben_da AS [FECHA DE INICIO DE LACTANCIA],
       cod_sex_bi AS [SEXO DEL BENEFICIARIO]
FROM Beneficiario
GO
```

```
USE pvl
GO
SELECT cod_ben_in [CÓDIGO DEL BENEFICIARIO],
       pat_ben_vc [APELLIDO PATERNO],
```

```
       mat_ben_vc [APELLIDO MATERNO],
       nom_ben_vc [NOMBRES DEL BENEFICIARIO],
       dni_ben_ch [DNI DEL BENEFICIARIO],
       fec_nac_ben_da [FECHA DE NACIMIENTO],
       fec_emb_ben_da [FECHA DE INICIO DE EMBARAZO],
       fec_lac_ben_da [FECHA DE INICIO DE LACTANCIA],
       cod_sex_bi [SEXO DEL BENEFICIARIO]
FROM Beneficiario
GO
```

```
USE pvl
GO
SELECT [CÓDIGO DEL BENEFICIARIO] = cod_ben_in,
       [APELLIDO PATERNO] = pat_ben_vc ,
       [APELLIDO MATERNO] = mat_ben_vc ,
       [NOMBRES DEL BENEFICIARIO] = nom_ben_vc ,
       [DNI DEL BENEFICIARIO] = dni_ben_ch ,
       [FECHA DE NACIMIENTO] = fec_nac_ben_da ,
       [FECHA DE INICIO DE EMBARAZO] = fec_emb_ben_da ,
       [FECHA DE INICIO DE LACTANCIA] = fec_lac_ben_da ,
       [SEXO DEL BENEFICIARIO] = cod_sex_bi
FROM Beneficiario
GO
```

| | CÓDIGO DEL BENEFICIARIO | APELLIDO PATERNO | APELLIDO MATERNO | NOMBRES DEL BENEFICIARIO | DNI DEL BENEFICIARIO | FECHA DE NACIMIENTO | FECHA DE INICIO DE EMBARAZO | FECHA DE INICIO DE LACTANCIA |
|---|---|---|---|---|---|---|---|---|
| 1 | 3 | ROSALES | RAVICHAHUA | SALOME | | 1930-10-12 | NULL | NULL |
| 2 | 4 | TUNQUI | DELGADO | VALENTIN | 08075067 | 1935-03-11 | NULL | NULL |
| 3 | 5 | CONTRERAS | FERNANDEZ | DINA LUCIA | 07437315 | 1948-05-30 | NULL | NULL |
| 4 | 7 | AYALA | CALDERON | YALITZA ANGELICA | | 2004-03-22 | NULL | NULL |
| 5 | 8 | TENA | AGUILAR | JULISSA | | 1998-05-12 | NULL | NULL |
| 6 | 9 | QUIROZ | FIESTAS | LUCIANO | 07066071 | 1932-03-02 | NULL | NULL |
| 7 | 10 | QUIROZ | SIERRA | EDU | | 2003-04-07 | NULL | NULL |
| 8 | 11 | GONZALES | BRAVO | MARCO | | 2002-06-10 | NULL | NULL |
| 9 | 13 | VIVAS | DELA CRUZ | ALISSON ELENA | | 1997-05-02 | NULL | NULL |
| 10 | 14 | BARREDA | RAMIREZ | JESSENIA | | 1993-12-18 | NULL | NULL |
| | 15 | RIVAS | VALLE | JUSTA | | 1943-01-08 | NULL | NULL |

```
SELECT CONVERT(INT, (POWER(2.0, 31) - 1)/3900000) AS [Cantidad de Biblias]
```

## Concatenación de cadenas

```
USE pvl
GO
SELECT cod_ben_in AS CÓDIGO,
       pat_ben_vc + ' ' + mat_ben_vc + ', ' + nom_ben_vc AS BENEFICIARIO,
       dni_ben_ch AS DNI,
       fec_nac_ben_da AS FECHA_NAC,
       fec_emb_ben_da AS FECHA_EMB,
```

```
       fec_lac_ben_da AS FECHA_LAC,
       cod_sex_bi AS SEXO
FROM Beneficiario
GO
```

```
USE pvl
GO
SELECT cod_ben_in AS CÓDIGO,
       CONCAT(pat_ben_vc, ' ', mat_ben_vc, ', ', nom_ben_vc) AS BENEFICIARIO,
       dni_ben_ch AS DNI,
       fec_nac_ben_da AS FECHA_NAC,
       fec_emb_ben_da AS FECHA_EMB,
       fec_lac_ben_da AS FECHA_LAC,
       cod_sex_bi AS SEXO
FROM Beneficiario
GO
```

```
USE pvl
GO
SELECT cod_ben_in AS CÓDIGO,
       CONCAT(pat_ben_vc, SPACE(1), mat_ben_vc, ', ', nom_ben_vc) AS BENEFICIARIO,
       dni_ben_ch AS DNI,
       fec_nac_ben_da AS FECHA_NAC,
       fec_emb_ben_da AS FECHA_EMB,
       fec_lac_ben_da AS FECHA_LAC,
       cod_sex_bi AS SEXO
FROM Beneficiario
GO
```

| | CÓDIGO | BENEFICIARIO | DNI | FECHA_NAC | FECHA_EMB | FECHA_LAC | SEXO |
|---|---|---|---|---|---|---|---|
| 1 | 3 | ROSALES RAVICHAHUA, SALOME | | 1930-10-12 | NULL | NULL | 1 |
| 2 | 4 | TUNQUI DELGADO, VALENTIN | 08075067 | 1935-03-11 | NULL | NULL | 1 |
| 3 | 5 | CONTRERAS FERNANDEZ, DINA LUCIA | 07437315 | 1948-05-30 | NULL | NULL | 0 |
| 4 | 7 | AYALA CALDERON, YALITZA ANGELICA | | 2004-03-22 | NULL | NULL | 0 |
| 5 | 8 | TENA AGUILAR, JULISSA | | 1998-05-12 | NULL | NULL | 0 |
| 6 | 9 | QUIROZ FIESTAS, LUCIANO | 07066071 | 1932-03-02 | NULL | NULL | 1 |
| 7 | 10 | QUIROZ SIERRA, EDU | | 2003-04-07 | NULL | NULL | 1 |
| 8 | 11 | GONZALES BRAVO, MARCO | | 2002-06-10 | NULL | NULL | 1 |
| 9 | 13 | VIVAS DELA CRUZ, ALISSON ELENA | | 1997-05-02 | NULL | NULL | 0 |
| 10 | 14 | BARREDA RAMIREZ, JESSENIA | | 1993-12-18 | NULL | NULL | 0 |
| 11 | 15 | RIVAS VALLE, JUSTA | | 1943-01-08 | NULL | NULL | 0 |
| 12 | 17 | LAZARO REYNOSO, BERNABE | 10580144 | 1901-01-01 | NULL | NULL | 1 |

## IIF(Condición, Verdadero, Falso)

La función IIF() en SQL Server es una función condicional que permite evaluar una expresión y devolver un valor basado en el resultado de esa evaluación.

Es similar a la función CASE, pero más compacta y fácil de usar en situaciones simples.

```
SELECT
      IIF([Title] IS NULL, '' , [Title] + ' ') + [FirstName] +
      IIF([MiddleName] IS NULL, '' , ' ' + [MiddleName]) + ' ' + [LastName] +
      IIF([Suffix] IS NULL, '' , ' ' + [Suffix]) AS FullName
FROM [AdventureWorks2025].[Person].[Person]
WHERE [Suffix] IS NOT NULL
```

| | FullName |
|---|---|
| 1 | Gary E. Altman III |
| 2 | Dan K Bacon Jr. |
| 3 | Mr. James T. Aguilar Jr. |
| 4 | Mr. Thomas B. Armstrong Jr. |
| 5 | Mr. James B. Bailey Jr. |
| 6 | Mr. Douglas A. Baldwin Jr. |
| 7 | Mr. Christopher R. Beck Jr. |
| 8 | Mr. Stanley A. Alan Jr. |
| 9 | Mr. Richard M. Bentley Jr. |
| 10 | Mr. Alexander J. Berger II |

```
SELECT
      IIF([Title] IS NULL, '' , [Title] + ' ') + [FirstName] +
      IIF([MiddleName] IS NULL, '' , ' ' + [MiddleName]) + ', ' + [LastName] +
      IIF([Suffix] IS NULL, '' , ' ' + [Suffix]) AS FullName
FROM [AdventureWorks2025].[Person].[Person]
WHERE [Title] IS NOT NULL
```

| | FullName |
|---|---|
| 100 | Mr. Chris N., Bidelman |
| 101 | Ms. Mary B., Billstrom |
| 102 | Mr. Jimmy, Bischoff |
| 103 | Mr. Mary, Bishop |
| 104 | Ms. Mae M., Black |
| 105 | Ms. Jackie E., Blackwell |
| 106 | Mr. Donald L., Blanton |
| 107 | Ms. Linda E., Burnett |
| 108 | Mr. Michael Greg, Blythe |
| 109 | Ms. Gabriel L., Bockenkamp |
| 110 | Mr. Michael L., Bohling |
| 111 | Ms. Corinna, Bolender |

```
USE pvl
GO
SELECT
    IIF(TRIM(pat_ben_vc) = '', '', pat_ben_vc + ' ') +
    mat_ben_vc + ', ' +
    nom_ben_vc AS BENEFICIARIO,
    dni_ben_ch AS DNI,
    fec_nac_ben_da AS FECHA_NAC,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO
FROM Beneficiario
GO
```

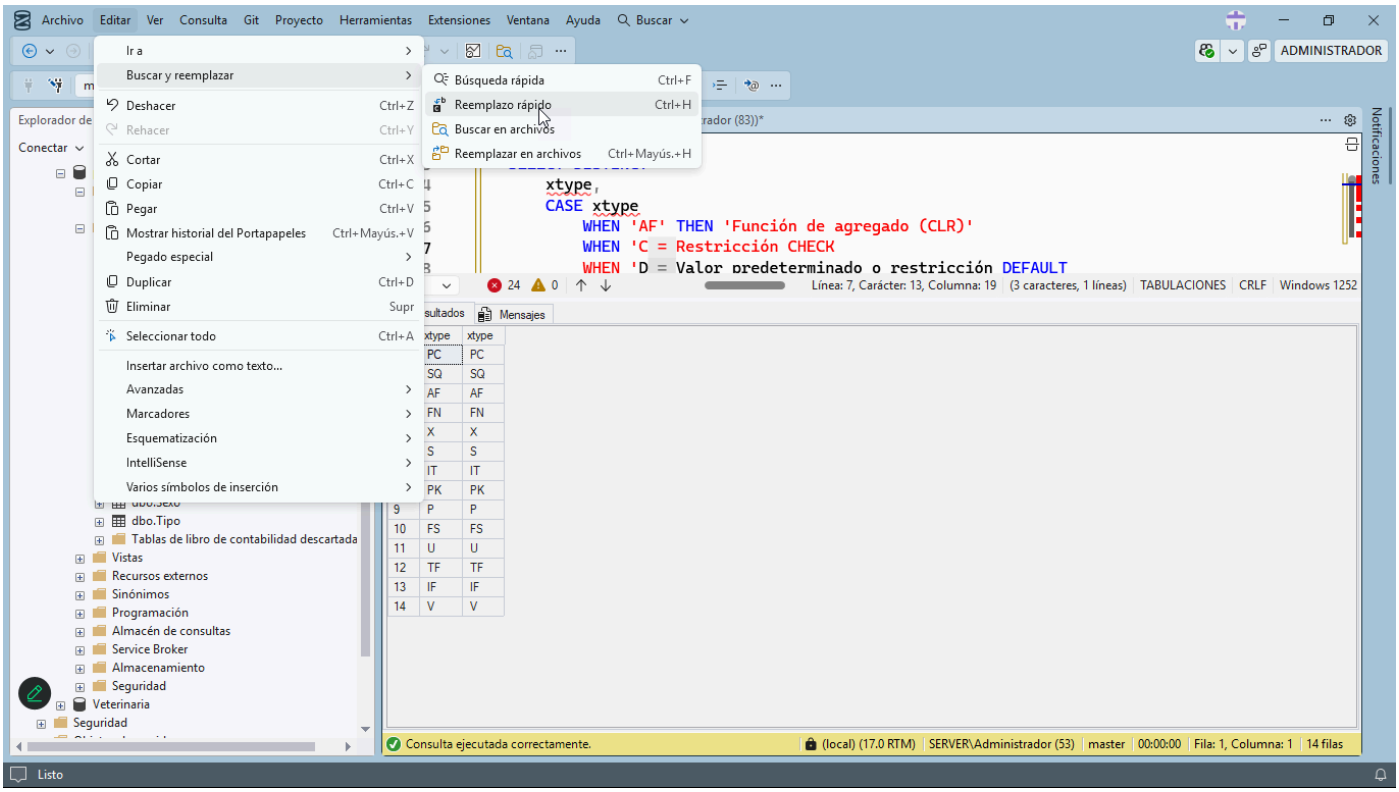
| | BENEFICIARIO | DNI | FECHA_NAC | FECHA_EMB | FECHA_LAC | SEXO |
|---|---|---|---|---|---|---|
| 1 | ROSALES RAVICHAHUA, SALOME | | 1930-10-12 | NULL | NULL | MASCULINO |
| 2 | TUNQUI DELGADO, VALENTIN | 08075067 | 1935-03-11 | NULL | NULL | MASCULINO |
| 3 | CONTRERAS FERNANDEZ, DINA LUCIA | 07437315 | 1948-05-30 | NULL | NULL | FEMENINO |
| 4 | AYALA CALDERON, YALITZA ANGELICA | | 2004-03-22 | NULL | NULL | FEMENINO |
| 5 | TENA AGUILAR, JULISSA | | 1998-05-12 | NULL | NULL | FEMENINO |
| 6 | QUIROZ FIESTAS, LUCIANO | 07066071 | 1932-03-02 | NULL | NULL | MASCULINO |
| 7 | QUIROZ SIERRA, EDU | | 2003-04-07 | NULL | NULL | MASCULINO |
| 8 | GONZALES BRAVO, MARCO | | 2002-06-10 | NULL | NULL | MASCULINO |
| 9 | VIVAS DELA CRUZ, ALISSON ELENA | | 1997-05-02 | NULL | NULL | FEMENINO |
| 10 | BARREDA RAMIREZ, JESSENIA | | 1993-12-18 | NULL | NULL | FEMENINO |
| 11 | RIVAS VALLE, JUSTA | | 1943-01-08 | NULL | NULL | FEMENINO |
| 12 | LAZARO REYNOSO, BERNABE | 10580144 | 1901-01-01 | NULL | NULL | MASCULINO |

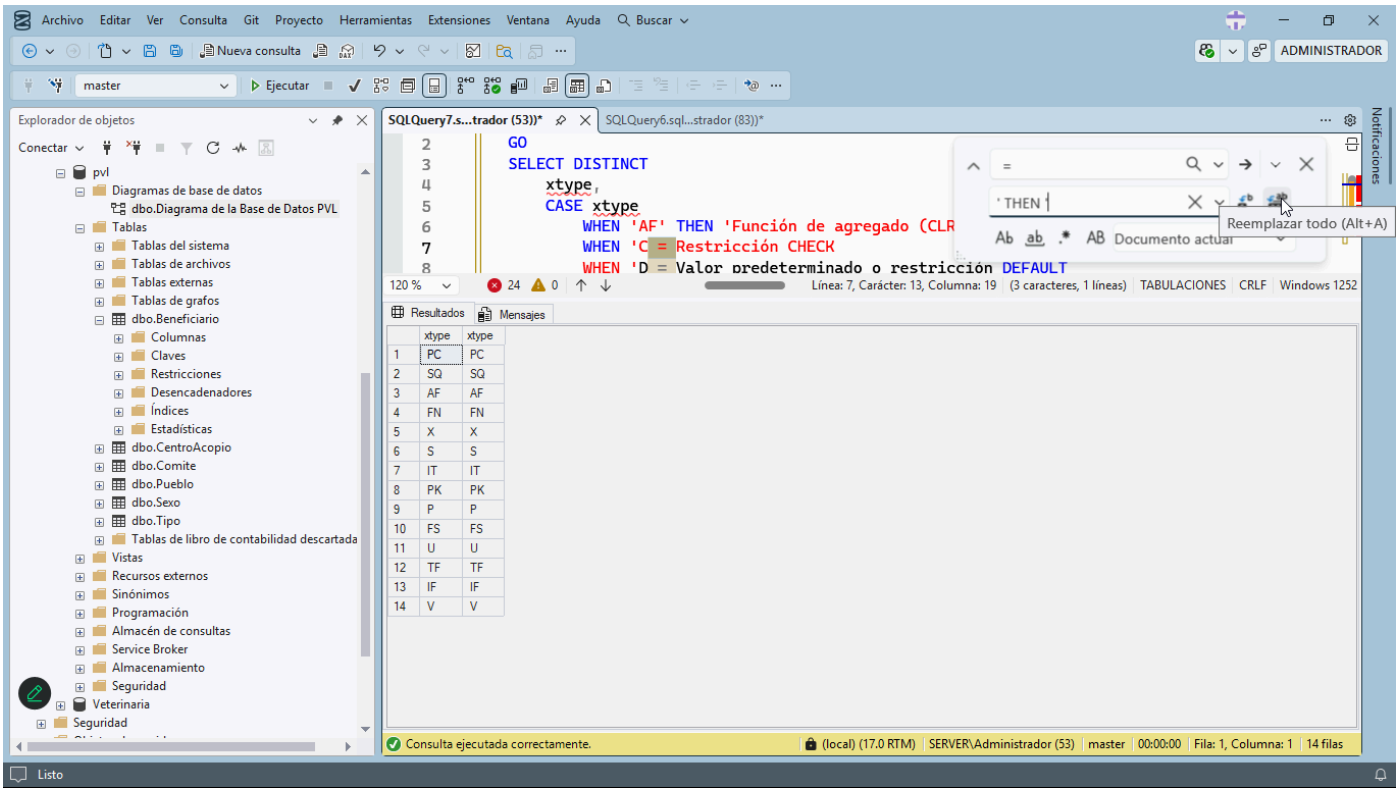
## CASE

La expresión CASE en SQL Server es una forma de manejar la lógica condicional dentro de las consultas.

Permite evaluar condiciones y devolver un valor basado en el resultado de esas evaluaciones.

Es útil cuando necesitas ejecutar más de una comparación y es más flexible que la función IIF().




```
USE master
GO
SELECT DISTINCT
    xtype AS Tipo,
    CASE xtype
        WHEN 'AF' THEN 'Función de agregado (CLR)'
        WHEN 'C' THEN 'Restricción CHECK'
        WHEN 'D' THEN 'Valor predeterminado o restricción DEFAULT'
        WHEN 'F' THEN 'Restricción FOREIGN KEY'
```

```
        WHEN 'L' THEN 'Registro'
        WHEN 'FN' THEN 'Función escalar'
        WHEN 'FS' THEN 'Función escalar del ensamblado (CLR)'
        WHEN 'FT' THEN 'Función con valores de tabla de ensamblado (CLR)'
        WHEN 'IF' THEN 'Función de tabla en línea'
        WHEN 'IT' THEN 'Tabla interna'
        WHEN 'P' THEN 'Procedimiento almacenado'
        WHEN 'PC' THEN 'Procedimiento almacenado del ensamblado (CLR)'
        WHEN 'PK' THEN 'Restricción PRIMARY KEY (de tipo K)'
        WHEN 'RF' THEN 'Procedimiento almacenado de filtro de replicación'
        WHEN 'S' THEN 'Tabla del sistema'
        WHEN 'SN' THEN 'Sinónimo'
        WHEN 'SO' THEN 'Secuencia'
        WHEN 'SQ' THEN 'Cola de servicio'
        WHEN 'TA' THEN 'Desencadenador DML del ensamblado (CLR)'
        WHEN 'TF' THEN 'Función de tabla'
        WHEN 'TR' THEN 'Desencadenador DML de SQL'
        WHEN 'TT' THEN 'Tipo de tabla'
        WHEN 'U' THEN 'Tabla de usuario'
        WHEN 'UQ' THEN 'Restricción UNIQUE (de tipo K)'
        WHEN 'V' THEN 'Vista'
        WHEN 'X' THEN 'Procedimiento almacenado extendido'
    END AS Descripción
FROM sysobjects
GO
```

| BENEFICIARIOS ZONAL URBANA | | | | | | |
|---|---|---|---|---|---|---|
| 48 | 49 | 50 | 51 | 52 | 54 | 56 |
| MENORES DE 1 ANO | NINOS DE 1 A 6 ANOS | MADRES GESTANTES | MADRES LACTANTES | 7 A 13 ANOS | ANCIANOS | TOTAL |
| | | | | | | |

# OPERADORES DE RELACIÓN O COMPARACIÓN

OPERADORES DE RELACIÓN O COMPARACIÓN

| Operador | Descripción |
|---|---|
| = | Igual a |
| > | Mayor que |
| < | Menor que |
| >= | Mayor o igual que |
| <= | Menor o igual que |
| <> | Diferente de |
| != | Diferente de |
| !> | No mayor que |
| !< | No menor que |
| ALL | Compara un valor escalar con un conjunto de valores de una sola columna. |

| Operador | Descripción |
|---|---|
| ANY / SOME | Compara un valor escalar con cualquier valor de un conjunto de valores de una sola columna. |
| BETWEEN | Indica si un valor está dentro de un rango especificado. |
| EXISTS | Se utiliza para comprobar la existencia de cualquier fila en una subconsulta. |
| IN | Compara un valor escalar con una lista de valores o el resultado de una subconsulta. |
| LIKE | Compara un valor de cadena con un patrón especificado. |
| NOT | Se utiliza para negar el resultado de una condición. |
| OR | Combina dos o más condiciones y devuelve verdadero si al menos una de las condiciones es verdadera. |
| IS NULL | Se utiliza para comprobar si un valor es NULL. |
| IS NOT NULL | Se utiliza para comprobar si un valor no es NULL. |

```
-- CALCULAR LA EDAD DE LOS BENEFICIARIOS AL 30 DE SETIEMBRE DE 2004
-- Y LISTARLOS CON SUS DATOS PERSONALES
USE pvl
GO
SELECT
    IIF(TRIM(pat_ben_vc) = '', '', pat_ben_vc + ' ') +
    mat_ben_vc + ', ' +
    nom_ben_vc AS BENEFICIARIO,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
GO
```

=

```
-- Mostrar los beneficiarios con 0 años de edad
USE pvl
GO
SELECT
```

```
    IIF(TRIM(pat_ben_vc) = '', '', pat_ben_vc + ' ') +
    mat_ben_vc + ', ' +
    nom_ben_vc AS BENEFICIARIO,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = 0
GO
```

```
-- En la base de datos deltron. Mostrar todos los productos cuyo precio de venta es de 10
USE deltron
GO
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo = 10
```

```
-- En la base de datos deltron. Mostrar todos los productos cuyo precio
-- de venta sea diferente a 10 dólares.
USE deltron
GO
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo != 10
ORDER BY pre_ven_prd_mo
```

```
-- En la base de datos deltron. Mostrar todos los productos cuyo precio
-- de venta sea diferente a 10 dólares.
```

```
USE deltron
GO
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo <> 10
ORDER BY pre_ven_prd_mo
```

```
-- Mostrar los beneficiarios mayores de 100 años
USE pvl
GO
SELECT
    IIF(TRIM(pat_ben_vc) = '', '', pat_ben_vc + ' ') +
    mat_ben_vc + ', ' +
    nom_ben_vc AS BENEFICIARIO,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) > 100
ORDER BY EDAD2 DESC
GO
```

```
-- En la base de datos deltron. Mostrar todos los productos
-- cuyo precio de venta sea menor a 2 dólares.
USE deltron
GO
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo < 2
ORDER BY pre_ven_prd_mo
```

```
-- En la base de datos deltron. Mostrar todos los productos
-- cuyo precio de venta sea >= 8000
USE deltron
GO
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo > 8000
ORDER BY pre_ven_prd_mo
```

```
-- En la base de datos deltron. Mostrar todos los productos
-- cuyo precio de venta sea <= 10
USE deltron
GO
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo <= 10
ORDER BY pre_ven_prd_mo
```

```
-- En la base de datos deltron. Mostrar todos los productos
-- cuyo precio de venta sea !< 8000
USE deltron
GO
SELECT cod_prd_vc, nom_prd_vc, pre_ven_prd_mo
FROM Almacen.producto
WHERE pre_ven_prd_mo !< 8000
ORDER BY pre_ven_prd_mo
```

```
-- En la base de datos deltron. Mostrar todos los productos
-- cuyo precio de venta sea entre 10 y 20 dólares
USE deltron
GO
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo BETWEEN 10 AND 20
ORDER BY pre_ven_prd_mo
```

```
-- En la base de datos deltron. Mostrar todos los productos
-- cuyo precio de venta NO sea entre 10 y 20 dólares
USE deltron
GO
```

```
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo NOT BETWEEN 10 AND 20
ORDER BY pre_ven_prd_mo
```

```
-- En la base de datos deltron. Mostrar todos los productos
-- cuyo precio de venta es de 10, 20, 30
USE deltron
GO
SELECT *
FROM Almacen.producto
WHERE pre_ven_prd_mo IN (10, 20, 30)
ORDER BY pre_ven_prd_mo
```

```
-- Mostrar los beneficiarios mayores de 100 años
USE pvl
GO
SELECT
    IIF(TRIM(pat_ben_vc) = '', '', pat_ben_vc + ' ') +
    mat_ben_vc + ', ' +
    nom_ben_vc AS BENEFICIARIO,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    fec_nac_ben_da AS FECHA_NAC,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE fec_nac_ben_da IN ('2004-06-01', '2004-06-03')
ORDER BY EDAD2 DESC
GO
```

```
USE master
GO
SELECT name AS 'Bases de Datos del Sistema'
FROM sysdatabases
WHERE name in ('master', 'model', 'msdb', 'tempdb')
```

```
USE master
GO
SELECT name AS 'Bases del Usuario'
FROM sysdatabases
WHERE name not in ('master', 'model', 'msdb', 'tempdb')
```

```
/*
    Mostrar los pedidos que no han sido
    entregados aún a nuestros clientes
*/
USE Northwind
GO
SELECT *
FROM Orders
WHERE ShippedDate IS NULL
GO
```

```
/*
    Mostrar los pedidos que han sido
    entregados aún a nuestros clientes
*/
USE Northwind
GO
SELECT *
FROM Orders
WHERE ShippedDate IS NOT NULL
GO
```

## LIKE

La cláusula LIKE en SQL Server se utiliza en la cláusula WHERE para buscar un patrón específico dentro de una columna de tipo texto.

Es particularmente útil para encontrar registros que coincidan con un patrón cuando no conoces el valor exacto.

## Caracteres Comodín

> - **El signo de porcentaje (%)**: Representa cero o más caracteres. Ejemplo: 'a%' buscará cualquier cadena que comience con 'a', como 'apple', 'armado', etc.
> - **El guion bajo (_)**: Representa exactamente un solo carácter. Ejemplo: 'a_c' buscará cualquier cadena de tres caracteres que comience con 'a' y termine con 'c', como 'abc', 'axc', etc.

```
/*
    Mostrar los beneficiarios
    cuyo apellido paterno comience con letra 'Y'
*/
USE pvl
GO
SELECT
    pat_ben_vc AS PATERNO,
    mat_ben_vc AS MATERNO,
    nom_ben_vc AS NOMBRES,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    fec_nac_ben_da AS FECHA_NAC,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE  pat_ben_vc LIKE 'Y%'
ORDER BY 1, 2, 3
GO
```

```
/*
    Mostrar los beneficiarios
    cuyo apellido paterno termine con letra 'Y'
*/
USE pvl
GO
SELECT
    pat_ben_vc AS PATERNO,
    mat_ben_vc AS MATERNO,
    nom_ben_vc AS NOMBRES,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    fec_nac_ben_da AS FECHA_NAC,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE  pat_ben_vc LIKE '%Y'
ORDER BY 1, 2, 3
GO
```

```
/*
    Mostrar los beneficiarios
    cuyo apellido paterno contengan la palabra 'AMOR'
*/
USE pvl
GO
SELECT
    pat_ben_vc AS PATERNO,
    mat_ben_vc AS MATERNO,
    nom_ben_vc AS NOMBRES,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    fec_nac_ben_da AS FECHA_NAC,
```

```
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE  pat_ben_vc LIKE '%AMOR%'
ORDER BY 1, 2, 3
GO
```

```
/*
    Mostrar los beneficiarios
    cuyo tercer caracter del  apellido paterno sea la letra 'M'
*/
USE pvl
GO
SELECT
    pat_ben_vc AS PATERNO,
    mat_ben_vc AS MATERNO,
    nom_ben_vc AS NOMBRES,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    fec_nac_ben_da AS FECHA_NAC,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
```

```
FROM Beneficiario
WHERE  pat_ben_vc LIKE '__M%'
ORDER BY 1, 2, 3
GO
```

```
/*
    Mostrar los beneficiarios
    cuyo tercer caracter del  apellido paterno sea la letra 'M', 'A', 'S'
*/
USE pvl
GO
SELECT
    SUBSTRING(pat_ben_vc, 3, 1) AS Caracter,
    pat_ben_vc AS PATERNO,
    mat_ben_vc AS MATERNO,
    nom_ben_vc AS NOMBRES,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    fec_nac_ben_da AS FECHA_NAC,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE  pat_ben_vc LIKE '__[MAS]%'
ORDER BY 1, 2, 3, 4
GO
```

```
/*
    Mostrar los beneficiarios
    cuyo tercer caracter del  apellido paterno sea de la letra M-S
*/
USE pvl
GO
```

```
SELECT
    SUBSTRING(pat_ben_vc, 3, 1) AS Caracter,
    pat_ben_vc AS PATERNO,
    mat_ben_vc AS MATERNO,
    nom_ben_vc AS NOMBRES,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    fec_nac_ben_da AS FECHA_NAC,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE  pat_ben_vc LIKE '__[M-S]%'
ORDER BY 1, 2, 3, 4
GO
```

```
/*
    Mostrar los beneficiarios
    cuyo tercer caracter del  apellido paterno NO sea de la letra M-S
*/
USE pvl
GO
SELECT
    SUBSTRING(pat_ben_vc, 3, 1) AS Caracter,
    pat_ben_vc AS PATERNO,
    mat_ben_vc AS MATERNO,
    nom_ben_vc AS NOMBRES,
    dni_ben_ch AS DNI,
    DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004  AS EDAD1,
    fec_nac_ben_da AS FECHA_NAC,
    CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)  AS EDAD2,
    fec_emb_ben_da AS FECHA_EMB,
    fec_lac_ben_da AS FECHA_LAC,
    IIF(cod_sex_bi = 0, 'FEMENINO', 'MASCULINO') AS SEXO,
    CASE
```

```
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) = (
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004) BE
        WHEN CONVERT(INT, DATEDIFF(DAY, fec_nac_ben_da, '2004-09-30') / 365.256363004)>= (
        ELSE 'ERROR'
    END AS [CICLO DE VIDA]
FROM Beneficiario
WHERE  pat_ben_vc LIKE '__[^M-S]%'
ORDER BY 1, 2, 3, 4
GO
```

## Limitación de Resultados

## TOP

La cláusula TOP en SQL se utiliza para limitar el número de filas que se devuelven en una consulta SELECT.

Es especialmente útil cuando se desea obtener un subconjunto de datos, como los primeros n registros de una tabla.

### Mis 5 productos con mayor cantidad en Stock

```
USE Northwind
GO
SELECT TOP 5
    ProductID AS Código,
    ProductName AS Producto,
    UnitsInStock AS Stock
FROM Products
ORDER BY UnitsInStock DESC;
```

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 75 | Rhönbräu Klosterbier | 125 |
| 2 | 40 | Boston Crab Meat | 123 |
| 3 | 6 | Grandma's Boysenberry Spread | 120 |
| 4 | 55 | Pâté chinois | 115 |
| 5 | 61 | Sirop d'érable | 113 |

## Mis 12 productos con menor cantidad en Stock

```
-- 12 Productos con menor cantidad en stock
USE Northwind
GO
SELECT TOP 12
    ProductID AS Código,
    ProductName AS Producto,
    UnitsInStock AS Stock
FROM Products
ORDER BY UnitsInStock ASC;
```

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 5 | Chef Anton's Gumbo Mix | 0 |
| 2 | 17 | Alice Mutton | 0 |
| 3 | 29 | Thüringer Rostbratwurst | 0 |
| 4 | 31 | Gorgonzola Telino | 0 |
| 5 | 53 | Perth Pasties | 0 |
| 6 | 21 | Sir Rodney's Scones | 3 |
| 7 | 66 | Louisiana Hot Spiced Okra | 4 |
| 8 | 74 | Longlife Tofu | 4 |
| 9 | 45 | Rogede sild | 5 |
| 10 | 8 | Northwoods Cranberry Sauce | 6 |
| 11 | 68 | Scottish Longbreads | 6 |
| 12 | 32 | Mascarpone Fabioli | 9 |

## Mis 6 productos con mayor cantidad en Stock

```
USE Northwind
GO
SELECT TOP 6
    ProductID AS Código,
    ProductName AS Producto,
    UnitsInStock AS Stock
FROM Products
ORDER BY UnitsInStock DESC;
```

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 75 | Rhönbräu Klosterbier | 125 |
| 2 | 40 | Boston Crab Meat | 123 |
| 3 | 6 | Grandma's Boysenberry Spread | 120 |
| 4 | 55 | Pâté chinois | 115 |
| 5 | 61 | Sirop d'érable | 113 |
| 6 | 33 | Geitost | 112 |

La cláusula **TOP WITH TIES** devuelve no solo las primeras N filas de un set de resultados, sino también todas las filas que tienen el mismo valor en la columna utilizada para la ordenación, en caso de que esos valores empaten.

```
USE Northwind
GO
SELECT TOP 6 WITH TIES
    ProductID AS Código,
    ProductName AS Producto,
    UnitsInStock AS Stock
FROM Products
ORDER BY UnitsInStock DESC;
```

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 75 | Rhönbräu Klosterbier | 125 |
| 2 | 40 | Boston Crab Meat | 123 |
| 3 | 6 | Grandma's Boysenberry Spread | 120 |
| 4 | 55 | Pâté chinois | 115 |
| 5 | 61 | Sirop d'érable | 113 |
| 6 | 33 | Geitost | 112 |
| 7 | 36 | Inlagd Sill | 112 |

```
USE Northwind
GO
SELECT TOP 5 PERCENT
    ProductID AS Código,
    ProductName AS Producto,
    UnitsInStock AS Stock
FROM Products
ORDER BY UnitsInStock DESC;
```

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 75 | Rhönbräu Klosterbier | 125 |
| 2 | 40 | Boston Crab Meat | 123 |
| 3 | 6 | Grandma's Boysenberry Spread | 120 |
| 4 | 55 | Pâté chinois | 115 |

## OFFSET/FETCH NEXT

La cláusula **OFFSET/FETCH NEXT** en SQL se utiliza para implementar la paginación en los resultados de una consulta.

Permite omitir un número determinado de filas y luego recuperar un número específico de filas.

Esta es una forma efectiva de manejar conjuntos de datos grandes y mostrar resultados de manera más estructurada.

```
USE Northwind
GO
SELECT
    ProductID AS Código,
    ProductName AS Producto,
    UnitsInStock AS Stock
FROM Products
ORDER BY UnitsInStock DESC
```

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 75 | Rhönbräu Klosterbier | 125 |
| 2 | 40 | Boston Crab Meat | 123 |
| 3 | 6 | Grandma's Boysenberry Spread | 120 |
| 4 | 55 | Pâté chinois | 115 |
| 5 | 61 | Sirop d'érable | 113 |
| 6 | 33 | Geitost | 112 |
| 7 | 36 | Inlagd Sill | 112 |
| 8 | 34 | Sasquatch Ale | 111 |
| 9 | 22 | Gustaf's Knäckebröd | 104 |
| 10 | 73 | Röd Kaviar | 101 |
| 11 | 46 | Spegesild | 95 |

```
USE Northwind
GO
SELECT
    ProductID AS Código,
    ProductName AS Producto,
    UnitsInStock AS Stock
FROM Products
ORDER BY UnitsInStock DESC
OFFSET 2 ROWS
```

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 6 | Grandma's Boysenberry Spread | 120 |
| 2 | 55 | Pâté chinois | 115 |
| 3 | 61 | Sirop d'érable | 113 |
| 4 | 33 | Geitost | 112 |
| 5 | 36 | Inlagd Sill | 112 |
| 6 | 34 | Sasquatch Ale | 111 |
| 7 | 22 | Gustaf's Knäckebröd | 104 |
| 8 | 73 | Röd Kaviar | 101 |
| 9 | 46 | Spegesild | 95 |
| 10 | 12 | Queso Manchego La Pastora | 86 |
| 11 | 41 | Jack's New England Clam Chowder | 85 |
| 12 | 59 | Raclette Courdavault | 79 |

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 6 | Grandma's Boysenberry Spread | 120 |
| 2 | 55 | Pâté chinois | 115 |
| 3 | 61 | Sirop d'érable | 113 |
| 4 | 33 | Geitost | 112 |
| 5 | 36 | Inlagd Sill | 112 |
| 6 | 34 | Sasquatch Ale | 111 |
| 7 | 22 | Gustaf's Knäckebröd | 104 |
| 8 | 73 | Röd Kaviar | 101 |
| 9 | 46 | Spegesild | 95 |
| 10 | 12 | Queso Manchego La Pastora | 86 |
| 11 | 41 | Jack's New England Clam Chowder | 85 |
| 12 | 59 | Raclette Courdavault | 79 |

```
USE Northwind
GO
SELECT
    ProductID AS Código,
    ProductName AS Producto,
    UnitsInStock AS Stock
FROM Products
ORDER BY UnitsInStock DESC
OFFSET 2 ROWS
FETCH NEXT 8 ROWS ONLY;
```

| | Código | Producto | Stock |
|---|---|---|---|
| 1 | 6 | Grandma's Boysenberry Spread | 120 |
| 2 | 55 | Pâté chinois | 115 |
| 3 | 61 | Sirop d'érable | 113 |
| 4 | 33 | Geitost | 112 |
| 5 | 36 | Inlagd Sill | 112 |
| 6 | 34 | Sasquatch Ale | 111 |
| 7 | 22 | Gustaf's Knäckebröd | 104 |
| 8 | 73 | Röd Kaviar | 101 |

# ACTIVIDAD IA

## 1. Seleccionar todos los empleados

```
USE Northwind
GO
SELECT *
FROM Employees;
```

## 2. Filtrar empleados por ciudad (igual a 'London')

```
USE Northwind
GO
SELECT EmployeeID, FirstName, LastName, City
FROM Employees
WHERE City = 'London';
```

## 3. Filtrar empleados por ID (mayor que 5)

```
USE Northwind
GO
SELECT EmployeeID, FirstName, LastName
FROM Employees
WHERE EmployeeID > 5;
```

## 4. Filtrar empleados por nombre (que comience con 'A')

```
USE Northwind
GO
SELECT EmployeeID, FirstName, LastName
FROM Employees
WHERE FirstName LIKE 'A%';
```

## 5. Filtrar empleados por fecha de nacimiento (anteriores a '1960-01-01')

```
USE Northwind
GO
SELECT EmployeeID, FirstName, LastName, BirthDate
FROM Employees
WHERE BirthDate < '1960-01-01';
```

## 6. Filtrar empleados donde la región sea NULL

```
USE Northwind
GO
SELECT EmployeeID, FirstName, LastName, Region
FROM Employees
WHERE Region IS NULL;
```

## 1. Buscar clientes cuyo nombre de compañía comience con 'A'

```
USE Northwind
GO
SELECT CompanyName, ContactName, City
FROM Customers
WHERE CompanyName LIKE 'A%';
GO
```

## 2. Buscar clientes cuyo nombre de contacto termine con 'o'